Java Backend Architecture

Interview Series

Chapter 12.10
Testing i18n Applications

50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

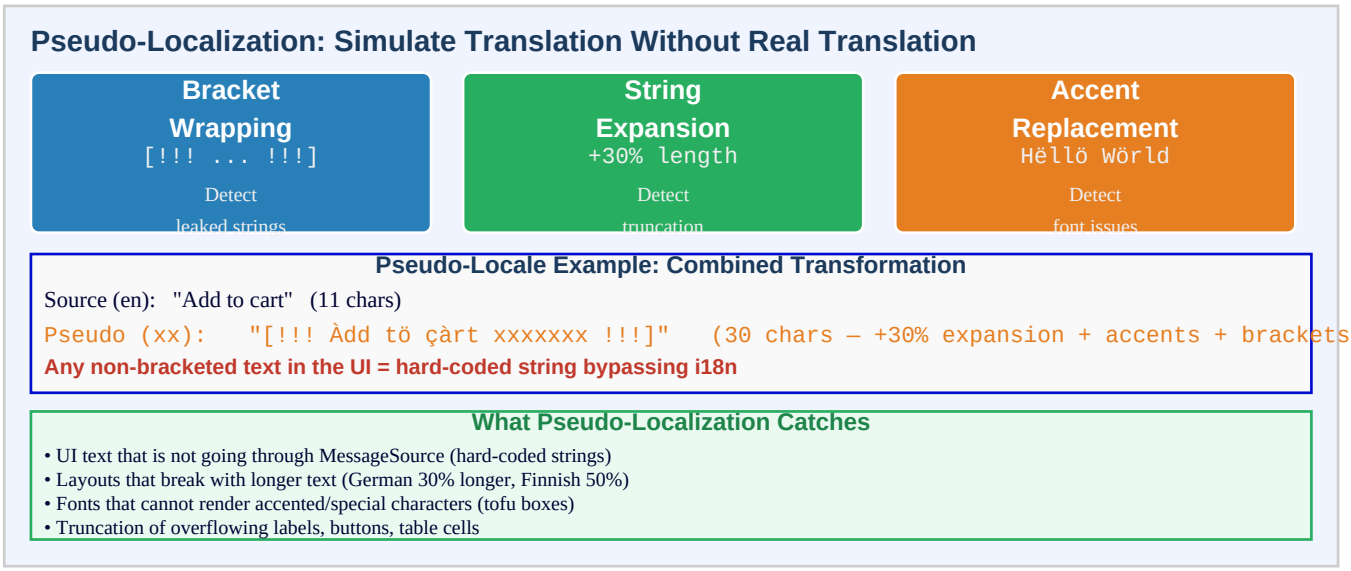
Chapter 12.10 – Testing i18n Applications

Introduction

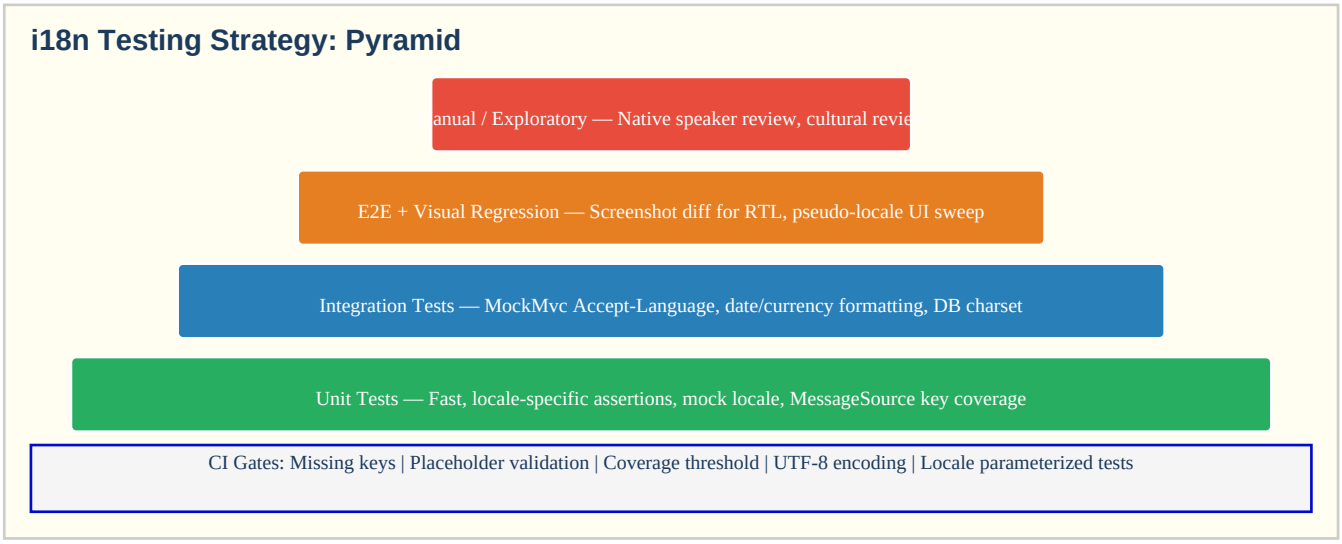
Testing internationalisation requires a distinct approach from standard functional testing. Pseudo-localization is a technique where the base locale strings are programmatically transformed — expanding length by 30%, adding accented characters, and wrapping strings in brackets — to simulate the visual effects of translation without requiring actual translations. This reveals hard-coded strings that bypass the message source, layouts that break with longer text, and fonts that cannot render accented characters. Locale-specific unit tests use Locale.setDefault() or inject a MockMvc request with an Accept-Language header to exercise formatting, validation messages, and date/currency display in specific locales.

A comprehensive i18n test suite covers multiple layers: unit tests for locale-sensitive business logic (date parsing, number formatting, message resolution), integration tests for the full request pipeline including locale detection and response serialization, and visual regression tests that capture screenshots for RTL locales and compare them to baselines to detect layout regressions. Unicode edge case testing is essential — test inputs should include emoji (4-byte UTF-8, surrogate pairs in Java), CJK characters, bidirectional mixed-text, Unicode normalization variants, and zero-width characters. Missing translation detection as a CI gate ensures no locale is ever deployed with untranslated strings visible to users.

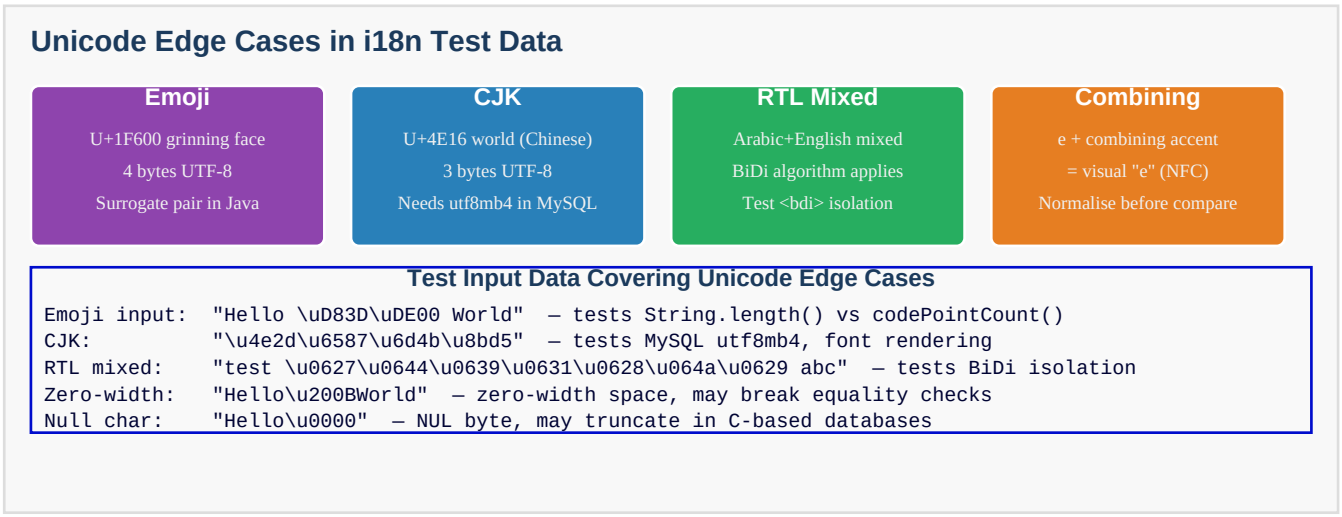
Pseudo-Localization Technique



i18n Test Strategy Pyramid



Unicode Edge Cases in Test Data



Key Definitions

Pseudo-localization	Transforming source strings programmatically (accents, brackets, length expansion) to simulate translation and detect i18n issues without actual translations
String expansion	Translation strings are longer than source; German is typically 30% longer than English; Finnish can be 50-60% longer; UI must accommodate this
Bracket wrapping	Surrounding pseudo-localized strings with [!!! ... !!!] so any unbracketed text in the UI is immediately identifiable as a hard-coded (non-i18n) string
Locale.setDefault()	Java method that sets the JVM-wide default locale; useful in tests but dangerous in production (shared JVM state); always restore after test execution
Accept-Language header	HTTP request header specifying preferred response language; MockMvc tests use .header("Accept-Language", "fr") to test locale detection
Visual regression test	Automated test that captures UI screenshots and compares them to approved baselines; detects layout regressions in RTL or long-text locales

Missing translation detection	CI check comparing keys in base locale (en) to every target locale; fails if any key is absent in a target locale
String truncation test	Testing that UI components (buttons, labels, cells) correctly display longer translated strings (especially German/Finnish) without clipping
ICU message format validation	Testing that ICU MessageFormat patterns ({count, plural, ...}) are syntactically correct and produce the expected output for edge counts (0, 1, 2, 100)
Parameterized locale test	JUnit @ParameterizedTest with @ValueSource of locale tags; runs the same test logic for each supported locale
Locale snapshot test	Asserting that a piece of formatted output (date, currency, number) exactly matches an expected value for a specific locale
Unicode normalization test	Testing that user inputs with different Unicode normalization forms (NFC vs NFD) compare equal after normalization
CJK test data	Test inputs containing Chinese, Japanese, Korean characters; tests multi-byte UTF-8, database charset, font support, and string length calculations
Emoji test data	Test inputs containing emoji (U+1F600 range); tests 4-byte UTF-8, surrogate pairs in Java String, MySQL utf8mb4, and String.length() vs codePointCount()
Spring Boot test slice	@WebMvcTest or @SpringBootTest with specific configurations; locale resolver, MessageSource, and LocaleContextHolder tested in isolation

i18n Testing Levels

Level	What to Test	Tools	Frequency
Unit	MessageSource keys, date/number formatting, plural logic	JUnit 5, Mockito	Every commit
Integration	MockMvc Accept-Language, DB charset, full pipeline	Spring Boot Test, MockMvc	Every commit
Visual Regression	RTL layout, text overflow, pseudo-locale UI	Playwright, Percy, Chromatic	Per PR
Missing Keys (CI)	All target locales have all keys from en base file	Custom JUnit / Gradle	Every commit
E2E	Full user flows in Arabic, German, Japanese	Playwright, Selenium	Per release
Manual	Native speaker review, cultural appropriateness	Human tester	Per locale release

Code Examples

1. Pseudo-localization implementation

```
// Pseudo-locale transformer: accents + brackets + expansion public class PseudoLocalizer { // Maps ASCII letters to accented equivalents private static final Map<Character, Character> ACCENT_MAP = Map.ofEntries( Map.entry('a', 'à'), Map.entry('e', 'é'), Map.entry('i', 'ï'), Map.entry('o', 'ø'), Map.entry('u', 'ü'), Map.entry('c', 'ç'), Map.entry('n', 'ñ'), Map.entry('A', 'À'), Map.entry('E', 'É'), Map.entry('I', 'Ï'), Map.entry('O', 'Ø'), Map.entry('U', 'Ü') ); public String pseudoLocalize(String source) { if (source == null || source.isBlank()) return source; StringBuilder sb = new StringBuilder(); for (char c : source.toCharArray()) { sb.append(ACCENT_MAP.getOrDefault(c, c)); } String accented = sb.toString(); // Expand by 30% by appending padding int expansionChars = Math.max(1, (int)(source.length() * 0.3)); String padding = "x".repeat(expansionChars); String expanded =
```

```
accented + " " + padding; // Wrap in brackets for visual detection of hard-coded strings return
"[!!! " + expanded + " !!!]"; } // Apply to an entire properties file to generate pseudo-locale
bundle public Properties pseudoLocalizeBundle(Properties source) { Properties pseudo = new
Properties(); for (String key : source.stringPropertyNames()) { String value =
source.getProperty(key); // Skip keys that should not be pseudo-localized (URLs, IDs, patterns)
if (isTranslatableValue(value)) { pseudo.setProperty(key, pseudoLocalize(value)); } else {
pseudo.setProperty(key, value); } } return pseudo; } private boolean isTranslatableValue(String
value) { // Don't pseudo-localize: URLs, {placeholder}-only, numbers return
!value.matches("https?://.*") && !value.matches("\{.*\}") && !value.matches("[0-9]+"); } } //
Test: assert pseudo-locale bundle has all keys from base locale @Test void
pseudoLocaleBundleHasAllKeys() { Properties en = loadProperties("messages_en.properties");
PseudoLocalizer localizer = new PseudoLocalizer(); Properties pseudo =
localizer.pseudoLocalizeBundle(en); assertThat(pseudo.stringPropertyNames())
.containsExactlyInAnyOrderElementsOf(en.stringPropertyNames()); }
```

2. Locale-specific unit tests with MockMvc

```
@SpringBootTest @AutoConfigureMockMvc class LocaleIntegrationTest { @Autowired MockMvc
mockMvc; // Test 1: locale from Accept-Language header affects response @ParameterizedTest
@MethodSource("localeAndGreeting") void greetingIsLocalized(String lang, String
expectedGreeting) throws Exception { mockMvc.perform(get("/api/greeting")
.header("Accept-Language", lang) .accept(MediaType.APPLICATION_JSON))
.andExpect(status().isOk())
.andExpect(jsonPath("$.message").value(containsString(expectedGreeting))); } static
Stream<Arguments> localeAndGreeting() { return Stream.of( Arguments.of("en", "Hello"),
Arguments.of("fr", "Bonjour"), Arguments.of("de", "Hallo"), Arguments.of("ar", "■■■■■") //
"■■■■■" ); } // Test 2: validation error messages are localized @Test void
validationErrorInFrench() throws Exception { mockMvc.perform(post("/api/users")
.header("Accept-Language", "fr") .contentType(MediaType.APPLICATION_JSON) .content("{"email":
"invalid-email"}")) .andExpect(status().isBadRequest())
.andExpect(jsonPath("$.errors[0].message") .value(containsString("valide"))); // French
validation message } // Test 3: date formatting is locale-specific @Test void
dateFormattedForGermanLocale() throws Exception { mockMvc.perform(get("/api/events/1")
.header("Accept-Language", "de-DE")) .andExpect(jsonPath("$.displayDate")
.value(matchesPattern("\d{2}\.\d{2}\.\d{4}"))); // German: dd.MM.yyyy } // Test 4: currency
formatted for Arabic (Saudi Arabia) @Test void currencyFormattedForArabicLocale() throws
Exception { mockMvc.perform(get("/api/products/1/price") .header("Accept-Language", "ar-SA"))
.andExpect(jsonPath("$.formattedPrice").isNotEmpty())
.andExpect(jsonPath("$.currency").value("SAR")); } // Test 5: RTL locale returns dir=rtl @Test
void arabicLocaleReturnsDirRtl() throws Exception { mockMvc.perform(get("/api/meta")
.header("Accept-Language", "ar")) .andExpect(jsonPath("$.direction").value("rtl")); } }
```

3. Missing translation detection and string truncation tests

```
@SpringBootTest class I18nCompletenessTest { private static final List<String>
SUPPORTED_LOCALES = List.of("fr", "de", "es", "ar", "he", "ja", "zh", "pt", "ko"); private
static final Map<String, Integer> STRING_MAX_LENGTHS = Map.of( "nav.menu.home", 12,
"checkout.button.place_order", 20, "user.registration.submit.button", 18 ); // Test 1: Missing
keys @Test void noMissingTranslationKeys() throws Exception { Properties base =
loadBundle("en"); Set<String> baseKeys = base.stringPropertyNames(); Map<String, Set<String>>
missing = new LinkedHashMap<>(); for (String locale : SUPPORTED_LOCALES) { Properties target =
loadBundle(locale); Set<String> missingKeys = baseKeys.stream() .filter(k ->
!target.containsKey(k)) .collect(Collectors.toCollection(TreeSet::new)); if
(!missingKeys.isEmpty()) missing.put(locale, missingKeys); } if (!missing.isEmpty()) { String
report = missing.entrySet().stream() .map(e -> e.getKey() + ": " + e.getValue().size() + "
missing keys") .collect(Collectors.joining(" ")); fail("Missing translations: " + report); } }
```

```
// Test 2: String length — German is typically 30% longer than English @Test void
germanStringsWithinAcceptableLength() throws Exception { Properties en = loadBundle("en");
Properties de = loadBundle("de"); for (Map.Entry<String, Integer> entry :
STRING_MAX_LENGTHS.entrySet()) { String key = entry.getKey(); int maxLength = entry.getValue();
String deValue = de.getProperty(key, ""); assertThat(deValue.length()) .as("German translation
of '%s' exceeds max length %d", key, maxLength) .isLessThanOrEqualTo(maxLength); } } // Test 3:
German strings are within 50% of English source length @Test void germanExpansionIsReasonable()
throws Exception { Properties en = loadBundle("en"); Properties de = loadBundle("de"); for
(String key : en.stringPropertyNames()) { String enVal = en.getProperty(key); String deVal =
de.getProperty(key, ""); if (!deVal.isEmpty() && enVal.length() > 5) { double ratio = (double)
deVal.length() / enVal.length(); assertThat(ratio) .as("German key '%s' is more than 2x longer
than English", key) .isLessThan(2.0); } } } private Properties loadBundle(String locale) throws
IOException { Properties p = new Properties(); String file = "messages" + (locale.equals("en")
? "" : "_" + locale) + ".properties"; try (InputStream is =
getClass().getClassLoader().getResourceAsStream(file)) { if (is != null) p.load(new
InputStreamReader(is, StandardCharsets.UTF_8)); } return p; } }
```

4. ICU MessageFormat validation and plural testing

```
import com.ibm.icu.text.MessageFormat; import com.ibm.icu.util.ULocale; class
IcuMessageFormatTest { // Test ICU plural forms for different locales @ParameterizedTest
@MethodSource("pluralTestCases") void pluralFormIsCorrectForLocale(String locale, int count,
String expected) { String pattern = loadMessage("product.count", locale); MessageFormat fmt =
new MessageFormat(pattern, new ULocale(locale)); String result = fmt.format(new
Object[]{count}); assertThat(result).isEqualTo(expected); } static Stream<Arguments>
pluralTestCases() { return Stream.of( // English: one / other Arguments.of("en", 0, "0
products"), Arguments.of("en", 1, "1 product"), Arguments.of("en", 2, "2 products"),
Arguments.of("en", 100, "100 products"), // Arabic: zero / one / two / few / many / other
Arguments.of("ar", 0, "■■ ■■■■■■"), // no products Arguments.of("ar", 1, "■■■■ ■■■■"), //
1 product Arguments.of("ar", 2, "■■■■■■■"), // 2 products // Russian: one / few / many
Arguments.of("ru", 1, "1 ■■■■■■"), // 1 ■■■■■■ Arguments.of("ru", 3, "3 ■■■■■■■"), // 3
■■■■■■■ Arguments.of("ru", 11, "11 ■■■■■■■■") // 11 ■■■■■■■■ ); } // Test ICU pattern syntax
validation @ParameterizedTest @ValueSource(strings = {"en", "fr", "de", "ar", "ja"}) void
icuPluralPatternsAreWellFormed(String locale) { List<String> icuKeys =
List.of("product.count", "cart.items", "notification.count"); for (String key : icuKeys) {
String pattern = loadMessage(key, locale); assertThatNoException() .describedAs("ICU pattern
for key '%s' locale '%s' is invalid", key, locale) .isThrownBy(() -> new MessageFormat(pattern,
new ULocale(locale))); } } private String loadMessage(String key, String locale) { Properties p
= new Properties(); // load messages_<locale>.properties // ... load logic return
p.getProperty(key, key); } }
```

5. Unicode edge case test data and String length assertions

```
class UnicodeEdgeCaseTest { // Test emoji in user input @Test void
emojiStoredAndRetrievedCorrectly() { String emoji = "■■"; // U+1F600 grinning face String name
= "User " + emoji + " Name"; // String.length() vs codePointCount
assertThat(emoji.length()).isEqualTo(2); // 2 char values (surrogate pair)
assertThat(emoji.codePointCount(0, emoji.length())).isEqualTo(1); // 1 Unicode char //
Database round-trip (requires utf8mb4 in MySQL) User saved =
userRepository.save(User.builder().name(name).build()); User retrieved =
userRepository.findById(saved.getId()).orElseThrow();
assertThat(retrieved.getName()).isEqualTo(name); // fails with MySQL utf8 (3-byte) } // Test
CJK characters @Test void cjkCharactersRoundTrip() { String cjk = "■■■■"; // "■■■■" (Chinese
"Chinese test") // 4 chars, each 3 bytes in UTF-8 = 12 bytes
assertThat(cjk.length()).isEqualTo(4);
assertThat(cjk.getBytes(StandardCharsets.UTF_8).length).isEqualTo(12); Content saved =
```

```
contentRepository.save(Content.builder().body(cjk).build());
assertThat(contentRepository.findById(saved.getId()).orElseThrow().getBody()) .isEqualTo(cjk);
} // Test Unicode normalization @Test void normalizedStringsCompareEqual() { // 'é' as
precomposed (U+00E9) vs decomposed (U+0065 U+0301) String precomposed = "é"; // é — one code
point String decomposed = "e■"; // e + combining accent — two code points // Without
normalization: strings are NOT equal assertThat(precomposed).isNotEqualTo(decomposed); //
After NFC normalization: equal String nfcA = Normalizer.normalize(precomposed,
Normalizer.Form.NFC); String nfcB = Normalizer.normalize(decomposed, Normalizer.Form.NFC);
assertThat(nfcA).isEqualTo(nfcB); // String length differs before normalization
assertThat(precomposed.length()).isEqualTo(1); assertThat(decomposed.length()).isEqualTo(2);
assertThat(nfcA.length()).isEqualTo(1); assertThat(nfcB.length()).isEqualTo(1); // normalized
} // Test zero-width space handling @Test void zeroWidthSpaceHandled() { String withZwsp =
"Hello■World"; // zero-width space (U+200B) assertThat(withZwsp.length()).isEqualTo(11);
assertThat(withZwsp.contains("■")).isTrue(); // Trimming and sanitization should handle
invisible characters String sanitized = withZwsp.replaceAll("\p{Cf}", ""); // remove format
chars assertThat(sanitized).isEqualTo("HelloWorld"); } // Test RTL mixed string with
bidirectional marks @Test void rtlMixedStringLength() { // Arabic text "■■■■■" (marhaba =
hello) — 5 code points, each 2 bytes in UTF-16 String arabic = "■■■■■";
assertThat(arabic.length()).isEqualTo(5); // 5 char values (all in BMP)
assertThat(arabic.codePointCount(0, arabic.length())).isEqualTo(5); // same, no surrogates
assertThat(arabic.getBytes(StandardCharsets.UTF_8).length).isEqualTo(10); // 2 bytes each } }
```

6. Spring Boot parameterized locale tests

```
@SpringBootTest @AutoConfigureMockMvc class ParameterizedLocaleTest { @Autowired MockMvc
mockMvc; @Autowired MessageSource messageSource; private static final List<String> ALL_LOCALES
= List.of("en", "fr", "de", "es", "ar", "he", "ja", "zh", "pt", "ko"); // Test 1: every
supported locale returns HTTP 200 for the home page @ParameterizedTest(name = "locale={0}")
@MethodSource("allSupportedLocales") void homePageLoadsForAllLocales(String lang) throws
Exception { mockMvc.perform(get("/") .header("Accept-Language", lang))
.andExpect(status().isOk()) .andExpect(content().encoding("UTF-8")); } // Test 2: no missing
message keys for any locale (MessageSource.getMessage does not throw) @ParameterizedTest(name =
"locale={0}") @MethodSource("allSupportedLocales") void
allMessageKeysResolveWithoutException(String lang) { Locale locale =
Locale.forLanguageTag(lang); List<String> keys = List.of( "nav.menu.home",
"user.registration.title", "error.validation.required", "success.registration.complete",
"checkout.button.place_order" ); for (String key : keys) { assertThatNoException()
.describedAs("Message key '%s' missing for locale '%s'", key, lang) .isThrownBy(() ->
messageSource.getMessage(key, new Object[]{"testField", 8}, locale)); } } // Test 3: date
formatting produces non-empty output for all locales @ParameterizedTest(name = "locale={0}")
@MethodSource("allSupportedLocales") void dateFormattingWorksForLocale(String lang) { Locale
locale = Locale.forLanguageTag(lang); LocalDate date = LocalDate.of(2025, 6, 15); String
formatted = DateTimeFormatter .ofLocalizedDate(FormatStyle.SHORT) .withLocale(locale)
.format(date); assertThat(formatted) .as("Date format for locale %s should not be empty", lang)
.isNotEmpty() .doesNotContain("null"); } // Test 4: currency formatting
@ParameterizedTest(name = "locale={0} currency={1}") @CsvSource({ "en-US, USD", "fr-FR, EUR",
"de-DE, EUR", "ar-SA, SAR", "ja-JP, JPY", "zh-CN, CNY" }) void currencyFormatsCorrectly(String
localeTag, String expectedCurrency) { Locale locale = Locale.forLanguageTag(localeTag);
Currency currency = Currency.getInstance(locale);
assertThat(currency.getCurrencyCode()).isEqualTo(expectedCurrency); } static Stream<String>
allSupportedLocales() { return ALL_LOCALES.stream(); } }
```

Interview Questions & Answers

Q1. What is pseudo-localization and what problems does it reveal?

Pseudo-localization transforms source (English) strings programmatically to simulate the visual effect of translation without actual translation. Three transformations: (1) Accent replacement: replace ASCII letters with accented equivalents (a→à, e→é). (2) String expansion: append padding to increase length by 30% (German is ~30% longer than English; Finnish can be 50-60% longer). (3) Bracket wrapping: surround with [!!! ... !!!] so any visible non-bracketed text is a hard-coded string bypassing i18n. Problems revealed: (a) Hard-coded strings not going through MessageSource. (b) Layouts that break with longer text (button overflow, label truncation). (c) Fonts that cannot render accented/special characters (tofu boxes). (d) String comparison logic that breaks with non-ASCII characters.

Q2. How do you test locale detection in a Spring Boot application with MockMvc?

Use MockMvc's .header('Accept-Language', 'fr') to simulate a French browser: mockMvc.perform(get('/api/data').header('Accept-Language', 'fr-FR')). Assert locale-specific behaviour: localized error messages (jsonPath('$.errors[0].message').value(containsString('obligatoire'))), formatted dates in French format, response Content-Language header. For cookie-based locale: perform a GET with a locale cookie, then assert the response uses the cookie locale. For URL parameter: GET /page?lang=de with LocaleChangeInterceptor. Parameterize across all supported locales with @ParameterizedTest + @MethodSource to ensure every locale returns 200 and doesn't throw MissingResourceException.

Q3. How do you assert that date and currency output is correct for a specific locale?

Date: format a known date with DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT) .withLocale(locale); assert it matches the locale's convention: en-US gives M/d/yy, de-DE gives d.M.yy (dot separator), fr-FR gives dd/MM/yyyy. For REST API output: assert the API returns ISO 8601 (2024-01-15T10:30:00Z) for machine-readable dates and a localized string for display fields. Currency: assert Currency.getInstance(locale).getCurrencyCode() equals the expected currency (en-US → USD, ja-JP → JPY); assert NumberFormat.getCurrencyInstance(locale) .format(1234.56) contains the correct symbol and format. Snapshot testing: store expected formatted strings in test resources; assert exact match — any localization library update or JDK update that changes formatting is caught.

Q4. What is visual regression testing for RTL layouts?

Visual regression testing captures screenshots of the application in specific locales and compares them pixel-by-pixel (or with perceptual hash) to previously approved baselines. For RTL: take screenshots with Accept-Language: ar and he; compare to approved RTL layout baselines. Tools: Percy (SaaS, browser-based), Playwright screenshot comparison (page.screenshot()), Chromatic (Storybook). What it catches: (1) Physical CSS properties (margin-left) that don't flip in RTL. (2) Icons that weren't mirrored. (3) Text overflow in narrow RTL buttons. (4) Flex layout that doesn't reverse in RTL. (5) Regression from a CSS change that accidentally broke RTL layout. Process: approve baselines once; CI fails if screenshots diverge; developer reviews the diff and either fixes or re-approves.

Q5. How do you test for missing translation keys in CI?

Write a JUnit test that: (1) Loads messages_en.properties (the base locale bundle). (2) Extracts the full Set of keys. (3) For each supported locale, loads messages_.properties. (4) For each key in the base bundle, verifies it exists in the locale bundle. (5) Fails with a detailed report listing missing keys by locale. Also check: (a) Empty values (key present but value = empty string). (b) Placeholder mismatches (source has {name} but target doesn't). (c) Orphaned keys (in target but not in base — stale translations). Configure a coverage threshold: allow up to 5% missing for newly added locales. This test runs on every commit and must pass before merge to main.

Q6. Why is German string length important for i18n testing?

German translations are typically 30-40% longer than equivalent English strings due to compound words and grammatical constructions. 'Place Order' (11 chars) might become 'Bestellung aufgeben' (20 chars — 82% longer). For UI components with fixed sizes (buttons, table headers, navigation items, mobile notifications), longer text may overflow, get truncated, or break the layout. Testing strategy: (1) Run pseudo-localization with 30% expansion to detect overflow before German translations are available. (2) Set character limits per key in the TMS; flag German translations that exceed the limit. (3) Integration test: check German translated buttons render correctly in screenshots. (4) In Spring: use Thymeleaf th:class='#{truncate}' for responsive handling.

Q7. How do you test ICU MessageFormat plural patterns across locales?

ICU MessageFormat supports CLDR plural rules with {count, plural, zero{} one{} few{} many{} other{}} patterns. Testing: (1) Syntax validation — new MessageFormat(pattern, locale) throws IllegalArgumentException if malformed. Parameterize across all locales: assert no exception for all plural keys. (2) Output correctness — test specific counts that hit each plural form: Arabic (ar): 0 (zero), 1 (one), 2 (two), 3-10 (few), 11-99 (many), 100 (other). Russian (ru): 1 (one), 3 (few), 11 (many). (3) Boundary conditions: Integer.MAX_VALUE, 0, -1, 1.5. (4) Assert placeholder preservation: {count} is in every plural form. (5) Edge: gender-specific forms in French — test both masculine and feminine forms.

Q8. How do you handle Locale.setDefault() in tests without causing side effects?

Locale.setDefault() changes a JVM-wide static setting. In JUnit 5 tests: (1) Use try-finally: Locale saved = Locale.getDefault(); try { Locale.setDefault(Locale.FRENCH); ... } finally { Locale.setDefault(saved); }. (2) JUnit 5 extension: create a @LocaleExtension that saves and restores the default locale for each test method. (3) Better: don't use Locale.setDefault() — pass explicit Locale to all methods under test. Avoid the static method. (4) Spring tests: use LocaleContextHolder.setLocale(locale) instead of Locale.setDefault() — LocaleContextHolder is request-scoped (ThreadLocal) and does not affect other threads. (5) If testing library code that uses Locale.getDefault(): use @ResourceLock from JUnit 5 to serialize tests that modify global state.

Q9. What Unicode edge cases should be included in i18n test data?

Test data should include: (1) Emoji: '\uD83D\uDE00' (U+1F600) — tests surrogate pairs in Java, 4-byte UTF-8, MySQL utf8mb4 requirement. (2) CJK: '\u4e2d\u6587' (Chinese) — 3-byte UTF-8, font support. (3) Combining characters: 'e\u0301' (é as e + combining accent) — tests Unicode normalization, equality, database storage. (4) RTL mixed: Arabic text embedded in English — tests BiDi algorithm, . (5) Zero-width space (U+200B): invisible character that breaks equality tests and search. (6) Directional marks (U+200F RTL mark): invisible directional control characters. (7) Null character (U+0000): may truncate strings in C-based databases. (8) Very long strings: test VARCHAR(255) boundary. (9) Strings with only whitespace or only combining characters.

Q10. How do you test Spring Boot test slices with locale context?

@WebMvcTest tests only the web layer (controllers, filters) — the service and repository layers are mocked. For i18n: (1) @WebMvcTest includes MessageSource and LocaleResolver beans from the application context by default. Test with MockMvc and Accept-Language header. (2) If testing a custom LocaleResolver: import it in @WebMvcTest(controllers = ...) or use @Import(LocaleConfig.class). (3) For @DataJpaTest (repository layer): locale doesn't directly affect JPA but test charset handling (save and retrieve UTF-8 strings with CJK/emoji). (4) For service-layer unit tests with @ExtendWith(SpringExtension.class): use LocaleContextHolder.setLocale() in @BeforeEach and reset in @AfterEach. (5) Full @SpringBootTest: use MockMvc with Accept-Language for end-to-end locale testing.

Q11. How do you test that locale propagation works correctly in @Async methods?

Strategy 1 — Synchronous executor for tests: override the async executor bean with @Primary @Bean that executes tasks synchronously (Runnable::run). The @Async method runs on the same thread; ThreadLocal

propagation is trivial. Test the business logic directly. Strategy 2 — Test the TaskDecorator: submit a task to the real executor; the task captures LocaleContextHolder.getLocale(); compare to the locale set before submitting. Use a CountDownLatch or CompletableFuture to wait. Strategy 3 — Integration test: LocaleContextHolder.setLocale(Locale.FRENCH); asyncService.process(); waitForCompletion(); assertThat(capturedLocale).isEqualTo(Locale.FRENCH); assertThat(LocaleContextHolder.getLocale()).isEqualTo(Locale.ENGLISH); // cleaned up.

Q12. What is a locale snapshot test and when do you use it?

A locale snapshot test stores the expected output for a specific locale as a reference file, then asserts exact equality on each test run. Useful for: (1) Number formatting: assertThat(formatted).isEqualTo('1.234,56 €') for de-DE — catches JDK updates that change formatting behaviour. (2) Date formatting: assertThat(formatted).isEqualTo('15.06.2025') for de-DE. (3) Pluralization output: assertThat(message).isEqualTo('3 Produkte') for de. Tooling: JUnit @SnapshotExtension or custom assertion that compares to a classpath resource. Maintenance: when JDK or ICU updates change the format legitimately, update the snapshot files explicitly (not automatically). Limitation: fragile — minor format changes break tests; use for stable, business-critical formatting assertions only.

Q13. How do you test character encoding in database I/O?

1. Repository test with emoji round-trip: save an entity with an emoji field (requires utf8mb4 in MySQL); assert the retrieved value equals the original. If MySQL charset is utf8 (3-byte), the test fails with DataIntegrityViolationException or with a corrupted string — exposing the misconfiguration. 2. CJK round-trip: save and retrieve Chinese/Japanese/Korean characters; assert exact match. 3. Large Unicode range: parameterize with strings from each Unicode block (Basic Latin, Latin Extended, CJK, Arabic, Hebrew, Devanagari). 4. VARCHAR length: assert that a 255-character BMP string fits in VARCHAR(255) but a 100-emoji string (200 chars in Java, 400 bytes in UTF-16, 400 bytes in utf8mb4) may exceed byte limits depending on the database column definition. 5. Use @DataJpaTest with an H2 in-memory database for fast charset unit tests.

Q14. How do you test that pseudo-locale strings are properly displayed in the UI?

1. Generate pseudo-locale bundle: run the PseudoLocalizer on messages_en.properties; save as messages_xx.properties. 2. Configure test environment: add 'xx' to supported locales. 3. Run the application with Accept-Language: xx. 4. Take screenshots with Playwright or Selenium. 5. Inspect screenshots for: (a) Any text NOT enclosed in [!!! ... !!!] — hard-coded string found. (b) Text that is truncated or overflows its container despite the 30% expansion. (c) Tofu boxes (■ characters) indicating missing glyph support. 6. Automate: a Playwright test that asserts page.locator('[data-hardcoded]') has no matches (after marking all i18n strings with a custom attribute). 7. Report: generate a list of hard-coded strings found; file as i18n tech debt.

Q15. How do you test placeholder preservation in translations?

1. Extract placeholder sets from source and target with regex \{(\w+)\}: Set sourcePh = extractPlaceholders(source); Set targetPh = extractPlaceholders(target); assertThat(targetPh).isEqualTo(sourcePh). 2. Test that MessageFormat.format() succeeds with realistic arguments: String msg = messageSource.getMessage('success.registration.complete', new Object[]{Map.of('name', 'Test User')}, locale); assertThat(msg).contains('Test User'). 3. Test with null argument: assert the template uses Optional or default value. 4. Test reordering: in some locales, arguments are in different order ({1} {0} instead of {0} {1}) — MessageFormat handles this, test that it works. 5. CI: run placeholder validation on every PR that touches .properties files.

Q16. What test infrastructure supports locale parameterization at scale?

1. JUnit 5 @ParameterizedTest with @MethodSource providing Stream: static Stream supportedLocales() { return SUPPORTED_LOCALES.stream().map(Locale::forLanguageTag); }. 2. Custom @LocaleTest annotation combining @ParameterizedTest + @MethodSource. 3. @CsvSource for locale + expected value pairs: @CsvSource({'fr, Bonjour', 'de, Hallo', 'ar, marhaba'}). 4. Spring @TestPropertySource: override spring.messages.basename for tests to use test-specific bundles with known values. 5. Test profiles: application-i18n-test.properties with spring.messages.cache-duration=0 to ensure bundle reload. 6. CI matrix: run the full test suite with LANG=fr_FR, LANG=ar_SA environment variables set to catch code that uses Locale.getDefault() instead of the application locale.

Q17. What are the key assertions to include in a comprehensive i18n test suite?

Functional: (1) All message keys resolve without MissingResourceException for all locales. (2) Number formatting matches locale convention (decimal/thousands separator). (3) Date formatting matches locale convention (dd/MM vs MM/dd vs yyyy-MM-dd). (4) Currency symbol and position correct per locale. (5) Plural forms correct for 0, 1, 2, few, many in each locale. Completeness: (6) No missing keys (CI gate). (7) No placeholder mismatches. (8) No empty translated values. Encoding: (9) Emoji and CJK round-trip correctly from DB. (10) UTF-8 response headers on all endpoints. Layout: (11) RTL locales return dir=rtl. (12) Visual regression passes for RTL screenshots. (13) German/Finnish strings fit within UI constraints (pseudo-locale expansion test). Security: (14) No SQL injection via locale parameter. (15) XSS-clean translated strings.

Q18. How do you test that a REST API handles locale-specific date input correctly?

Scenario: an API accepts a date in the user's locale format. Test strategy: (1) Send an ISO 8601 date in the request body with the correct Content-Type and Accept-Language header; assert the stored date is correct. (2) Send a locale-formatted date without ISO 8601 — assert the API rejects it with a clear error (or parses it if the API supports locale-aware input). (3) Test edge cases: Feb 29 in a leap year, Dec 31, Jan 1 across year boundary. (4) Test timezone: send 2024-01-15T23:30:00-05:00 (New York); assert stored UTC is 2024-01-16T04:30:00Z. (5) Test invalid date: 2024-02-30 should return 400 with a localized error message in the request's Accept-Language. (6) Parameterize across locales: the API must reject locale-formatted dates (14/01/2024) and only accept ISO 8601 — assert 400 for locale-format inputs.

Q19. What should a comprehensive i18n CI pipeline include?

Stage 1 — Static analysis (every commit): (a) Missing translation key check (messages_en keys present in all target locales). (b) Placeholder validation (source and target placeholders match). (c) UTF-8 encoding validation of all .properties files. (d) No duplicate keys within any single bundle. (e) No orphaned keys (in target but not in base). Stage 2 — Unit/integration tests: (f) Locale-parameterized unit tests for formatting. (g) MockMvc tests with Accept-Language for all supported locales. (h) Emoji/CJK database round-trip tests. Stage 3 — Visual regression (per PR): (i) Screenshot diff for RTL locales (ar, he). (j) Pseudo-locale UI sweep screenshot. Stage 4 — Release gate: (k) TMS export: only approved translations included in the release build. (l) Coverage report: total key coverage per locale published as build artifact.